A Comprehensive analysis of the CBP 2025 Branch Predictors entries: LVCP, RVA-Toru, and TAGE-SC 2025

Saumya Patel

March 25, 2026

Contents

1 Introduction

In the previous assignment, I evaluated the following branch predictors:

- Load Value Correlator Predictor (LVCP)
- Register-Value-Aware Branch Predictor (RVA-Toru)
- TAGE-SC 2025 André Seznec Predictor
- TAGE-SC-L Alberto Ros Predictor

In this report, I will continue my analysis of these predictors by performing a similar analysis of the remaining CBP2025 entries, namely

- MultiPerspective Perceptron Predictor (MPP) by Jimenez, et al.
- Programming Idiom Predictor (PIP)
- Branch Prediction via Load Value Prediction (BALL) by Jun Fan
- The Bullseye Predictor by Emet Behrendt, et al.

I will do a similar analysis of these predictors as I did for the first four and then I will compare all 8 predictors together to draw a final conclusion and rank them based on their performance.

1.1 The Oracle Predictor

The Oracle predictor is a theoretical construct that represents the best possible branch predictor that is not ahead of time. The point of this predictor is to check the maximum possible performance of a branch predictor on the given traces. It is important to note that the Oracle predictor is not a real predictor and cannot be implemented in hardware, but it serves as a useful benchmark for evaluating the performance of other predictors. More implementation details about the Oracle predictor can be found in the later sections of this report.

2 Methodology

The methodology for evaluating the predictor is the same as the one used in the previous assignment. I will be using the CBP2025 traces to evaluate the performance of the predictors. I will be using the same metrics as before, namely

- Branches Per Cycles (BrPerCyc)
- Misprediction Branches Per Cycles (MispBrPerCyc)
- Misprediction Rate (MR)
- Misprediction Rate Per Kilo Instructions (MPKI)
- Wrong Path Cycles (CycWP)
- Wrong Path Cycles Per Kilo Instructions (CycWPKI)

Furthermore, I also compared the predictors against each other to draw a final conclusion about their performance. The metrics that I used were,

- **Delta Misprediction Per Kilo Instructions:** $\Delta_{MPKI} = \text{MPKI}_A - \text{MPKI}_B$
  where A and B are two predictors being compared. This metric gives us a direct comparison of the misprediction rates of the two predictors.

3 MultiPerspective Perceptron Predictor (MPP)

3.1 Details

The Paper argues that traditional predictors look at branch history from two angles, the global and the local history. Jimenez argues that there are many useful "perspectives" on the same history. He introduces the MPP which is essentially a hashed perceptron that combines many of these perspectives together. The new features that this predictor introduces are the following:

- **MODHIST (Modulo History):** The author describes that he noticed a problem that he called ***branch misalignment*** i.e. if branch B sometimes appears in the history and some does not, then the predictor will have a hard time learning the pattern, because the same outcome sequence would land at different positions in the history register. This problem has seemed to break traditional and hashed predictors. To solve this problem, the author introduces MODHIST which essentially only records branches whose addresses are divisible by some modulos, filtering out the "misaligned" branches and thus making the history more consistent.

- **RECENCYPOS:** It tracks a stack of recently executed branches using LRU policy. knowing where the branch is in this stack can be useful to correlate the outcome of the branch with its recency.

- **BLURRYPATH:** It records coarser memory regions instead of exact addresses. A "coarser" memory region is defined as regions of memory who have the first N bits of their address the same. For example, if you take the address *0x7FFF4A28* and right shift it by 4 bits you would get *0x7FFF4A*, which would mean that every address that only differs in those last 4 bits would be considered the same "blurry" address.

  You would want to track these "blurry" addresses because of 2 reasons, **first**, they have less noise. If your program takes a slightly different path through the same general code region, precise history says "completely different." Blurry history says "same region, close enough." This can actually generalize better because you're capturing the structural shape of execution rather than its exact fingerprint. **Second**, compression. Fewer distinct values means less aliasing pressure in the hash tables which means that different histories are less likely to collide on the same table entry.